LEARN2COMPILE

# Frontend Foundation & Team Development Standard

*EAMCET Preparation Platform • Version 1.0*

Purpose: establish one shared approach to Angular architecture, routing, reusable components, API integration, testing, naming, and Git collaboration. All frontend contributors should follow this document and discuss changes to shared conventions before introducing alternatives.

## 1. Core principles

- Feature-first: product functionality belongs in its feature folder.
- core = app-wide infrastructure; shared = generic reusable UI; layouts = page shells; features = product behavior.
- Keep components focused. Move API calls into services and business logic into suitable services or pure functions.
- Create folders when there is a real need; avoid empty folders and unnecessary abstractions.
- Build responsive, accessible UI with keyboard support, semantic HTML and visible focus states.
- Never put secrets in frontend code. The backend must enforce authorization, payment verification and exam scoring.

## 2. Recommended repository structure

```
L2C-rankforge-Emcet/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── core/          # app-wide auth, config, guards, interceptors, services
│   │   │   ├── shared/        # generic reusable UI, pipes, directives, validators
│   │   │   ├── layouts/       # public, student/app and admin shells
│   │   │   ├── features/      # product features
│   │   │   ├── app.routes.ts
│   │   │   ├── app.config.ts
│   │   │   └── app.*          # retain filenames generated by this Angular version
│   │   ├── assets/
│   │   ├── main.ts
│   │   └── styles.scss
│   ├── angular.json
│   ├── package.json
│   └── README.md
├── backend/                   # separate Node.js/Express application
├── docs/                       # architecture, API contracts, Git and coding standards
└── README.md
```

Angular versions differ. Do not rename generated bootstrap files just to match examples. If core/features/layouts/shared currently sit directly under src/, move them under src/app/, update imports and run the build.

## 3. Folder responsibilities

| Folder | What belongs there |
|---|---|
| **core/** | App-wide authentication state, API configuration, guards, interceptors and global services. No page-specific UI. |
| **shared/** | Reusable generic components such as buttons, dialogs, loaders, empty/error states, pipes and directives. |
| **layouts/** | Public, student and admin shells with shared navigation and routed content. |
| **features/** | Feature-specific pages, components, services, models and route definitions. |
| **assets/ + styles.scss** | Static images/fonts and global styles/design tokens. |
| **docs/** | Architecture decisions, setup instructions, API contracts, naming and Git workflow. |

## 4. Feature folder pattern

```
features/practice/
├── pages/              # route-level screens
├── components/         # practice-only UI
├── services/           # API calls and feature logic
├── models/             # feature-specific TypeScript types
├── utils/              # feature-only helpers, when needed
└── practice.routes.ts # routes owned by this feature
```

Use this pattern for substantial features such as learning, practice, mock-tests, results and subscriptions. Small features can remain simple; do not create empty folders for symmetry.

## 5. Feature map

| Feature | Responsibility |
|---|---|
| **home** | Public landing and product information |
| **auth** | Login, registration, recovery and verification UI |
| **dashboard** | Student overview and recent activity |
| **syllabus** | Subjects, chapters and topics navigation |
| **learning** | Topic pages, notes and formulas |
| **practice** | Practice sessions, answer selection and explanations |
| **mock-tests** | Instructions, timer, exam interface and submission flow |
| **results** | Scores, accuracy, answer review and performance |
| **profile** | Student details and preferences |
| **subscription/payments** | Plans and checkout/status UI; server verifies payments |
| **admin** | Content and user management UI; server enforces permissions |
| **not-found** | 404 and unknown-route handling |

## 6. Routing standards

- Keep root route configuration in src/app/app.routes.ts.
- Let each substantial feature own its routes in a file such as practice.routes.ts.
- Lazy-load larger feature routes where appropriate.
- Use layouts to wrap route groups; do not duplicate navigation on every page.

- Guards improve navigation but do not replace backend authorization.
- Include sensible redirects and a not-found route.

## 7. Naming and code conventions

- Use kebab-case filenames and follow the naming generated by the installed Angular CLI.
- Use PascalCase for classes and types (PracticeService, QuestionDto); camelCase for variables and functions (loadQuestions()).
- Use TypeScript types/interfaces for API contracts; avoid any unless justified.
- Keep API requests in services rather than scattering HttpClient calls across components.
- Use consistent imports and avoid introducing path aliases without updating project configuration.
- Comments should explain why a non-obvious decision exists, not restate obvious code.

## 8. Design system and UI standards

- Define global color, typography, spacing, radius, shadow and breakpoint tokens in the project's chosen styling system.
- Create shared buttons, inputs, cards, dialogs, progress indicators and loading/empty/error states as needed.
- Use shared components for genuinely common UI; keep feature-specific UI inside its feature.
- Data-driven screens should handle loading, success, empty and error states.
- Use semantic HTML, accessible labels, keyboard navigation and visible focus states.
- Design mobile-first and verify narrow mobile, tablet and desktop widths.

## 9. API and security standards

- Angular communicates with Express through HttpClient; it never connects directly to MongoDB.
- Use one API base URL configuration and agree on a versioned prefix such as /api/v1.
- Document request/response formats in docs/api-contracts.md before parallel frontend/backend work.
- Handle network, validation, unauthorized and server errors consistently.
- Never commit secrets or put private keys/database credentials in Angular environment files.
- The backend is authoritative for user roles, payment status, protected content, answer evaluation and final scores.
- Use the team's agreed authentication mechanism consistently; do not invent separate token storage behavior per feature.

## 10. Testing and quality gates

- Check frontend/package.json for the available build, lint and test scripts; run the configured checks before a pull request.
- Test navigation, form validation, loading/error states and critical student journeys.
- Add tests for important business logic and reusable components where practical.
- Never merge code that breaks the build or exposes credentials.

## 11. Git collaboration

- Keep main stable; work on focused feature/fix branches.
- Pull the latest main before starting and resolve conflicts carefully.

- Use descriptive commits, e.g. feat(practice): add answer selection.
- Open a pull request with purpose, test evidence and screenshots for UI changes.
- Have a teammate review changes before merging where practical.
- Push only to the repository/branch where you have permission. Never force-push shared main.

**Example workflow:**

```
git switch main
 git pull <team-remote> main
 git switch -c feature/practice-question-card
 git add .
 git commit -m "feat(practice): add question card"
 git push -u <team-remote> feature/practice-question-card
```

Replace <team-remote> with the team's approved remote name, and confirm you have write access before pushing.

## 12. Definition of done

- Code is in the correct feature/shared folder and follows naming conventions.
- Responsive behavior and relevant loading, empty and error states are handled.
- API contract is agreed and no secrets are committed.
- Configured build and relevant tests pass.
- Pull request explains the change and how it was verified.

## 13. Recommended implementation order

1. Audit the generated Angular structure, version and available npm scripts.
2. Move core/shared/layouts/features under src/app/ if currently outside it; fix imports and verify build.
3. Create minimal public, student and admin layout shells plus route groups.
4. Define design tokens and a small set of shared UI components.
5. Establish API configuration and consistent HttpClient/error handling.
6. Initialize Express and a health endpoint in parallel.
7. Build one complete vertical slice: syllabus → question → answer → result.
8. Expand into mock tests, authentication, progress, subscriptions and admin after the first slice is stable.

## 14. Team decision rule

Start with the simplest structure that serves the feature. Promote code into core/shared only when it is truly app-wide or reused. Discuss changes to routing, authentication, API contracts, global styling and shared components in a pull request so the team stays aligned.

Before treating this as final, confirm the current Angular version, SSR choice, styling approach, authentication approach and approved Git remote/branch permissions.